

CCF-GESP C++四级

编程能力等级认证



第七课

二维数组 1

01. 二维数组的定义和访问



知识讲解

二维数组



二维数组可以被看作是一种**特殊的一维数组**：

—— 它的每个**元素**都是一个**一维数组**。

列 (二维)

a[0]	1	2	3	4	行 (一维)
a[1]	5	6	7	8	
a[2]	9	10	11	12	

二维数组a



知识讲解

二维数组的定义和初始化



格式：数据类型 数组名[行数][列数]；

```
int a[3][4];
```

```
int a[3][4] = { {1,2,3,4}, {5,6,7,8}, {9,10,11,12} };
```

```
int a[][4] = { {1,2,3,4},{5,6,7,8},{9,10,11,12} };
```

a[0]	1	2	3	4
a[1]	5	6	7	8
a[2]	9	10	11	12

二维数组的**行数**可以省略，计算机可以自动确定数组a的行数。



知识讲解

二维数组的部分初始化

- **部分初始化：**未被初始化的部分自动填充0。

a[0]	1	2	3	4
a[1]	5	6	0	0
a[2]	0	0	0	0

```
int a[3][4] = {  
    {1,2,3,4},  
    {5,6}  
};
```

- **省略行合并部分初始化：**自动推断行，未被初始化的部分自动补0。

a[0]	1	2	3	4
a[1]	5	6	0	0
a[2]	9	10	0	0

```
int a[][4] = {{1,2,3,4},{5,6},{9,10}};
```



知识讲解

访问二维数组元素

- 通过**行下标**和**列下标**访问二维数组中的元素。

		列下标			
		0	1	2	3
行下标	0	1	2	3	4
	1	5	6	7	8
	2	9	10	11	12

```
cout<<a[1][2];
```

```
a[2][0]=18;
```



知识讲解

二维数组的遍历

按行遍历

```
for (int i = 0; i < m; i++) {  
    for (int j = 0; j < n; j++)  
        cout << a[i][j] << " ";  
    cout << endl;  
}
```

- 外层循环遍历行
- 内层循环遍历列

按列遍历

```
for (int j = 0; j < n; j++) {  
    for (int i = 0; i < m; i++)  
        cout << a[i][j] << " ";  
    cout << endl;  
}
```

- 外层循环遍历列
- 内层循环遍历行



真题解析

判断题

(样题) 已知数组 a 定义为 `int a[100];`, 则赋值语句 `a['0'] = 3;` 会导致编译错误。

☐ 正确

错误 ☒



真题解析

判断题

(2023年6月) 如果希望记录 10 个最长为 99 字节的字符串, 可以将字符串数组定义为 `char s[100][10];`。

☐ 正确

错误 ☒



真题解析

判断题

(2023年9月) 如果希望记录10个最长为99字节的字符串, 可以将字符串数组定义为 `char s[10][100];` 。

☒ 正确

错误 ☐



真题解析

选择题

(样题) 以下哪个选项能正确定义一个二维数组 (**D**)

- A. `int a[][];`
- B. `char b[][4];`
- C. `double c[3][];`
- D. `bool d[3][4];`



真题解析

选择题

(样题) 以下哪个选项能正确访问二维数组 array 的元素 (**C**)

- A. array[1, 2]
- B. array(1)(2)
- C. array[1][2]
- D. array{1}{2}



真题解析

选择题

(2023年6月) 执行以下 C++ 语言程序后, 输出结果是 (**D**)

```
1  #include <iostream>
2  using namespace std;
3  int main() {
4      int array[3][3];
5      for (int i = 0; i < 3; i++)
6          for (int j = 0; j < 3; j++)
7              array[i][j] = i * 10 + j;
8      int sum;
9      for (int i = 0; i < 3; i++)
10         sum += array[i][i];
11     cout << sum << endl;
12     return 0;
13 }
```

- A. 3
- B. 30
- C. 33
- D. 无法确定



知识讲解

二维数组做函数参数



传递多维数组时，只有形式参数的**第一维**的长度可以**省略**，形式参数的**其他维**的长度都不能省略。

```
void printEle(int arr[][4]){  
    for(int i=0;i<4;i++)  
        for(int j=0;j<4;j++)  
            cout<<arr[i][j]<<' '  
}  
int main(){  
    int arr[4][4] = {0};  
    printEle(arr);  
    return 0;  
}
```

传递地址

实参



真题解析

选择题

(样题) 以下哪个函数声明在调用时可以传递二维数组的名字作为参数 (**A**)

- A. `void BubbleSort(int a[3][4]);`
- B. `void BubbleSort(int a[][]);`
- C. `void BubbleSort(int * a[]);`
- D. `void BubbleSort(int ** a);`



真题解析

选择题

(2023年6月) 以下哪个函数声明在调用时可以传递二维数组的名字作为参数 (**A**)

- A. `void BubbleSort(int a[][4]);`
- B. `void BubbleSort(int a[3][]);`
- C. `void BubbleSort(int * a[]);`
- D. `void BubbleSort(int ** a);`

02 二维数组的存储



知识讲解

二维数组的存储



二维数组的元素在内存中是是**连续存放**的，其本质也是按照**一维数组**的方式存储；

```
int a[3][4];
```

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

按行存放，每行中的元素是**按列下标**从**小到大**的顺序存放。



知识讲解

二维数组的存储



二维数组其本质也是按照一维数组的方式访问元素，
需要根据所访问的元素行下标计算出对应的下标

行下标 × 数组列长度 + 列下标

1	2	3	4	5	6	7	8	9	10	11	12
a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]
0	1	2	3	4	5	6	7	8	9	10	11

```
int a[3][4] = {  
    {1,2,3,4},  
    {5,6,7,8},  
    {9,10,11,12}  
};  
cout<<a[1][2];
```

$a[1 \times 4 + 2] \rightarrow a[6]$

7



知识讲解

二维数组的存储



二维数组其本质也是按照一维数组的方式访问元素，
需要根据所访问的元素行列下标计算出对应的下标

行下标 $\times$ 数组列长度 + 列下标

1	2	3	4	5	6	7	8	9	10	11	12
a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]
0	1	2	3	4	5	6	7	8	9	10	11

列下标范围0~3

```
int a[3][4] = {  
    {1,2,3,4},  
    {5,6,7,8},  
    {9,10,11,12}  
};  
cout<<a[0][5]; 6
```

$a[0 \times 4 + 5] \rightarrow a[5]$

没有越界!!!



知识讲解

二维数组的存储



二维数组其本质也是按照一维数组的方式访问元素，
需要根据所访问的元素行下标计算出对应的下标

行下标 $\times$ 数组列长度 + 列下标

1	2	3	4	5	6	7	8	9	10	11	12
a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]
0	1	2	3	4	5	6	7	8	9	10	11

```
int a[3][4] = {  
    {1,2,3,4},  
    {5,6,7,8},  
    {9,10,11,12}  
};
```

a[3][1];



越界!!!

a[2][5];



越界!!!



知识讲解

二维数组所占内存



二维数组所占内存大小，是**全部元素**所占内存之和。

1	2	3	4	5	6	7	8	9	10	11	12
a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[2][0]	a[2][1]	a[2][2]	a[2][3]
0	1	2	3	4	5	6	7	8	9	10	11

```
int a[3][4] = {  
    {1,2,3,4},  
    {5,6,7,8},  
    {9,10,11,12}  
};  
cout<<sizeof(a);
```

$4 \times 3 \times 4 \rightarrow 48$

48



真题解析

选择题

(2023年6月) 下列关于 C++ 语言中数组的叙述,
不正确的是(D)

- A. 一维数组在内存中一定是连续存放的。
- B. 二维数组是一维数组的一维数组。
- C. 二维数组中的每个一维数组在内存中都是连续存放的。
- D. 二维数组在内存中可以不是连续存放的。



真题解析

选择题

(样题) 如果有如下二维数组定义, 则 `a[0][3]` 的值为 (**C**)

```
int a[2][2] = {{0, 1}, {2, 3}};
```

- A. 编译出错
- B. 1
- C. 3
- D. 0



真题解析

选择题

(2023年6月) 一个二维数组定义为 `double array[3][10];`, 则这个二维数组占用内存的大小为 (**D**)

- A. 30
- B. 60
- C. 120
- D. 240



真题解析

选择题

(2023年9月) 一个二维数组定义为 `char array[3][10];` , 则这个二维数组占用内存的大小为 (**B**)

- A. 10
- B. 30
- C. 32
- D. 48



真题解析

选择题

(2023年6月) 一个二维数组定义为 `int array[5][3];`, 则 `array[1][2]` 和 `array[2][1]`, 在内存中的位置相差多少字节 ()

- A. 2 字节
- B. 4 字节
- C. 8 字节
- D. 无法确定



真题解析

思路分析

`int array[5][3];`

- ① 先计算`array[1][2]`和`array[2][1]`在内存里相差几个元素;
 - `array[1][2]`在一维数组中的下标: $1 \times 3 + 2 = 5$
 - `array[2][1]`在一维数组中的下标: $2 \times 3 + 1 = 7$
 - 相差元素的个数: $7 - 5 = 2$
- ② 再计算相差的字节数: **int字节数** × **相差的元素个数**
 - $4 \times 2 = 8$



真题解析

选择题

(2023年6月) 一个二维数组定义为 `int array[5][3];`, 则 `array[1][2]` 和 `array[2][1]`, 在内存中的位置相差多少字节 (**C**)

- A. 2 字节
- B. 4 字节
- C. 8 字节
- D. 无法确定